

A Comparative Study of Encodings to Design Combinational Logic Circuits Using Particle Swarm Optimization

Carlos A. Coello Coello, Erika Hernández Luna
CINVESTAV-IPN
Evolutionary Computation Group
Dpto. de Ing. Elect./Secc. Computación
Av. IPN No. 2508, Col. San Pedro Zacatenco
México, D.F. 07300, MEXICO
ccoello@cs.cinvestav.mx
eluna@computacion.cs.cinvestav.mx

Arturo Hernández Aguirre
Center for Research in Mathematics (CIMAT)
Department of Computer Science
Guanajuato, Gto. 36240, MEXICO
artha@cimat.mx

Abstract

This paper extends our original proposal to use Particle Swarm Optimization (PSO) to design combinational logic circuits [4] in which a binary representation was adopted. In this case, we study the impact of the representation adopted. For that sake, we adopt 2 integer representations (one of which is proposed by us) and we compare them with respect to our previous binary representation and with respect to a multiobjective genetic algorithm that uses an integer encoding. For our comparative study, we adopted several combinational logic circuits of one and several outputs whose designs have been previously studied in the specialized literature. Our results indicate that PSO can be a competitive algorithm for circuit design when using one of the integer representations proposed.

1 Introduction

The Particle Swarm Optimization algorithm was originally proposed by Kennedy and Eberhart [7, 8] and has its roots in artificial life, social psychology, engineering and computer science. The main idea behind PSO is to simulate the movement of a flock of birds seeking food. In this simulation, the behavior of each individual gets affected by both an individual and a social factor. Each individual contains its current position in the search space as well as its velocity and the best position found by the individual so far [8].

PSO can be seen as a distributed behavioral algorithm that performs (in its more general version) multidimensional search. In the simulation, the behavior of each individual is affected by either the best local (i.e., within a certain neighborhood) or the best global individual. The ap-

proach uses a population of potential solutions (called “particles”) and a measure of performance similar to the fitness value used with evolutionary algorithms. Also, the adjustments of individuals are analogous to the use of a crossover operator. However, this approach introduces the use of flying potential solutions through hyperspace (used to accelerate convergence). Additionally, PSO allows individuals to benefit from their past experiences [8].

The past success of PSO in diverse optimization tasks motivated us to use this heuristic to design combinational circuits [4]. However, the main issue when applying PSO to combinational circuit design has to do with the encoding adopted. In our previous work, we used binary representation and we were able to find reasonably good results, but without outperforming in any way to a genetic algorithm [4]. In this paper, we adopt 2 integer representations for the use of PSO in the design of combinational logic circuits (one of which is proposed by us), and we compare them. The main goal of this work is to study the impact of the representation adopted on the performance of our PSO algorithm.

2 Description of our Approach

The problem of interest to us consists of designing a circuit that performs a desired function (specified by a truth table), given a certain specified set of available logic gates. In circuit design, one can use various criteria to define minimal-cost expressions. The complexity of a logic circuit is a function of the number of gates in the circuit. The complexity of a gate generally is a function of the number of inputs to it. Because a logic circuit is a realization (implementation) of a Boolean function in hardware, reducing the number of literals in the function should reduce the num-

ber of inputs to each gate and the number of gates in the circuit—thus reducing the complexity of the circuit. Therefore, the main objective of the work reported here is to minimize the total number of gates of feasible circuits (i.e., those that match all the desired outputs expressed in their truth table).

To encode a circuit, we used the same matrix representation adopted in some of our previous work [2]. Such representation is shown in Figure 1. More formally, we can say that any circuit can be represented as a bidimensional array of gates $S_{i,j}$, where j indicates the *level* of a gate, so that those gates closer to the inputs have lower values of j . (Level values are incremented from left to right in Figure 1). For a fixed j , the index i varies with respect to the gates that are “next” to each other in the circuit, but without being necessarily connected. Each matrix element is a gate (there are 5 types of gates: AND, NOT, OR, XOR and WIRE¹) that receives its 2 inputs from any gate at the previous column as shown in Figure 1. This sort of encoding was originally proposed by Louis [9]. The so-called “cartesian genetic programming” [11] also adopts a similar encoding to the matrix previously described.

A logic circuit can be encoded using integers that correspond to the type of gate and its inputs. PSO, however, tends to deal with either binary or real-numbers representation. For our comparative study, we will adopt two integer representations: (1) **Integer A** (proposed by Hu et al. [5]), and (2) **Integer B** (proposed by us in this paper).

In the PSO algorithm, the individual factor P_{best} refers to the decisions that the individual has made so far and that have worked best (in terms of its performance measure). This value has an impact on its future decisions. Additionally, the social factor N_{best} refers to the decisions that the other individuals (within a certain neighborhood) have made so far and that have worked best for them. This value will also affect the future decisions of the individuals in the given neighborhood.

Figure 2 shows the pseudocode of the PSO algorithm that we propose for the design of combinational logic circuits. Its main difference with respect to traditional PSO has to do with the update of the position of the particle in each of its dimensions (which is marked with ** in Figure 2). Both, the **Integer A** and the **Integer B** approaches normalize the velocity of each dimension of the particle in the range 0 to 1, so that we can further determine (in a random way using a flip function **F**) whether we need to change the current position or not (this is done with the probability given by the velocity). If the change is required, then we copy to the particle the value of N_{best} in the current position. Otherwise, the **Integer A** approach leaves the particle intact. When

¹WIRE basically indicates a null operation, or in other words, the absence of gate, and it is used just to keep regularity in the representation used, since otherwise would have to use variable-length strings.

```

Randomly initialize the population of particles,  $P$ .
Repeat {
  For each particle  $i$  in the population  $P$  {
    Compute the fitness of the particle  $P[i]$ 
    If the fitness of  $P[i]$  is better than the fitness of
    the best particle found so far  $P_{best}[i]$ ,
    Then update  $P_{best}$  using  $P[i]$ 
  }
  For each particle  $i$  in  $P$  {
    Select the particle with the best fitness in the
    topological neighborhood of  $P[i]$ 
    and update the value of  $N_{best}[i]$ 
  }
  For each particle  $i$  in the population  $P$  {
    Compute the new velocity for each dimension of
    the particle
     $\vec{V}[i]_{new} = \vec{V}[i]_{old} + \phi_1(\vec{P}_{best}[i] - \vec{P}[i]) +$ 
     $\phi_2(\vec{N}_{best}[i] - \vec{P}[i])$ 
    ** Update the position of the particle  $P[i]$ 
  }
  Apply uniform mutation with a (user given) rate.
} Until reaching the stop condition

```

Figure 2. Pseudocode of the PSO algorithm adopted in this work. Note the addition of a mutation operator.

the change is not required, the **Integer B** approach checks again whether is necessary to change the current position, but now using a probability of $1 - v_d$, where v_d is the current velocity. If the change is required, then we copy to the particle the value of P_{best} in the position that we are updating. Otherwise, we leave the particle intact. These two integer representations are exemplified in Figure 3. As in our previous work [4], we introduce here a mutation operator to our PSO algorithm in order to improve its exploratory power, since this seems necessary when applying this approach to the design of circuits. Furthermore, in this case, the particles try to follow the same characteristics of N_{best} and P_{best} and could get stuck in their current position. Thus, the use of a mutation operator is vital in order to avoid this problem. Our fitness function works in two stages. At the beginning of the search, only validity of the circuit outputs is taken into account, and the algorithm is basically exploring the search space. Once a functional solution appears, then the fitness function is modified such that any valid designs produced are rewarded for each WIRE gate that they include, so that the algorithm tries to find the circuit with the maximum number of WIRES that performs the function required.

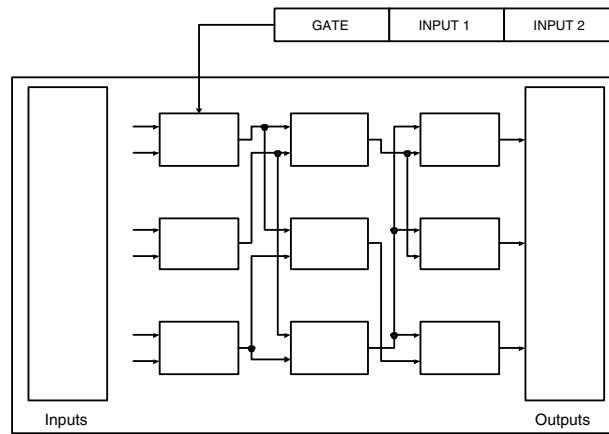


Figure 1. Matrix used to represent a circuit. Each gate gets its inputs from either of the gates in the previous column. Note the encoding adopted for each element of the matrix as well as the set of available gates used.

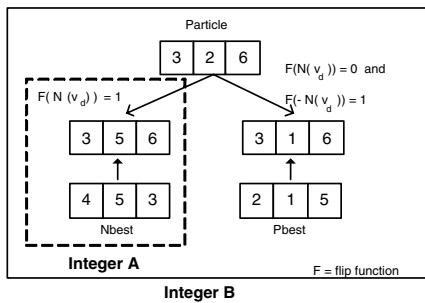


Figure 3. Example of the two integer representations used for our PSO algorithm.

3 Comparison of Results

For the comparative study reported in this paper, we adopted several circuits of different degrees of complexity. Only 5 of them will be reported in this paper due to space limitations. We compared the following algorithms:

- **MGA:** A multi-objective genetic algorithm that uses a population-based approach [3]. This approach has been found to be highly competitive with respect to other genetic algorithms, which motivated our desire to use it in this comparative study.
- **EAPSO:** PSO with the **Integer A** representation (this is the encoding proposed in [5]).
- **EBPSO:** PSO with the **Integer B** representation (this encoding is proposed by us in this paper).

- **BPSO:** PSO with binary representation.

For our test problems, we used examples reported in other references as well as some files available in the *Berkeley Logic Interchange Format* (blif) that have been used for optimization of multi-level logical networks of nodes [14]. For each of the examples reported, we performed 20 independent runs using the following set of available gates: AND, OR, NOT, XOR and WIRE. The size of the matrix adopted in all cases was 5×5 , except for the third example, in which a matrix size 6×6 was adopted. The parameters adopted for BPSO were: $\phi_1 = \phi_2 = 0.8$, $V_{max} = 3.0$, mutation rate (P_m) = 0.1 and size of the neighborhood = 3. EAPSO and EBPSO used: $\phi_1 = \phi_2 = 0.2$, $V_{max} = 0.4$, $P_m = 0.1$ and size of the neighborhood = 3. The MGA used a mutation rate of 0.00667 and a crossover rate of 0.5.

3.1 Example 1

Our first example has 4 inputs and 1 output as can be seen in Table 1. In this case, BPSO, EAPSO and EBPSO used 50 particles and 2000 iterations (i.e., a total of 100,000 fitness function evaluations). The MGA used a population size of 170 and 600 generations (i.e., 102,000 fitness function evaluations). The comparison of results is shown in Table 2. It can be seen that all the algorithms compared were able to converge to a feasible solution in 90% of the runs performed, but only EBPSO reached a rate of 100%. Indeed, EBPSO converged to the best solution found for this circuit, which has only 6 gates (this solution is depicted in Figure 4) during 85% of the time (the highest success rate of all the approaches compared). Both EAPSO and BPSO found this solution 75% of the time and the MGA found it only 30% of

the time. EBPSO had the best average performance in this case, with an average fitness of 34.85 and an average number of gates (of the final solutions found) of 6.15 (a value very close to the best solution found). The second best algorithm was BPSO, with an average fitness of 34.25 and an average number of gates of 7.30. It is worth noticing that the MGA was the worst performer in this example, finding the best solution only 30% of the time, and reaching feasible solutions only in 90% of the runs. It should be noted that the best solution found by a human designer for this circuit had 10 gates.²

D	C	B	A	S
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	1
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	1
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

Table 1. Truth table of the first example.

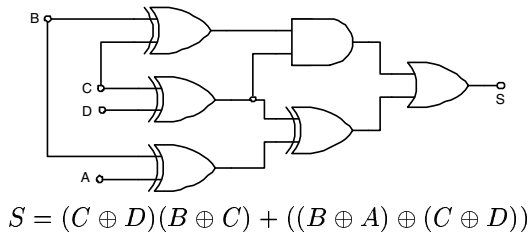


Figure 4. Graphical representation of the best circuit found for example 1.

3.2 Example 2

Our second example has 4 inputs and 1 output, as indicated in Table 3. In this case, BPSO, EAPSO and EBPSO

²The human designer used a combination of Karnaugh Maps [6], the Quine-McCluskey method [10, 12] and the theorems of Boolean algebra to reach the solutions reported in this paper.

	BPSO	EAPSO	EBPSO	MGA	HD
# of gates of best sol.	6	6	6	6	10
Freq. of best sol.	75%	75%	85%	30%	-
Feas. circs.	100%	95%	100%	90%	-
Avg. # gates	6.75	7.30	6.15	9.30	-
Avg. fitness	34.25	33.70	34.85	31.70	-
Std. Dev.	1.6181	4.4615	0.3664	6.2669	-

Table 2. Comparison of results between our three PSO algorithms (EBPSO, EAPSO, BPSO), the multiobjective genetic algorithm (MGA), and a human designer (HD) for example 1.

used 50 particles and 2000 iterations (i.e., a total of 100,000 fitness function evaluations). The MGA used a population size of 170 and ran for 600 generations (i.e., a total of 102,000 fitness function evaluations). The comparison of results is shown in Table 4. We can see that in this case, all the algorithms were able to find as their best solution a feasible circuit with 5 gates (this circuit is depicted in Figure 5). However, EBPSO was the only approach that found the best solution in more than half of the runs (55% of the time). EBPSO was also the only approach that was able to find feasible circuits in 100% of the runs. Finally, EBPSO also had the best average number of gates (5.90, which is very close to the best solution found) and the best average fitness (35.10). The second best algorithm in this case, was EAPSO, with an average fitness of 32.75 and an average number of gates of 8.25. Note that again, the MGA was the worst performer in this example, finding the best solution only 10% of the time, and reaching feasible solutions only in 70% of the runs. The best solution found by a human designer for this circuit had 10 gates (twice the number of the best solution found by any of the heuristics compared).

3.3 Example 3

Our third example has 5 inputs and 1 output, as indicated in Table 5. In this case, we adopted a matrix of size 6×6 . BPSO, EAPSO and EBPSO used 50 particles and 10,000 iterations (i.e., a total of 500,000 fitness function evaluations). The MGA used a population size of 330 and 1600 generations (i.e., a total of 528,000 fitness function evaluations). The comparison of results for this example is shown

D	C	B	A	S
0	0	0	0	0
0	0	0	1	0
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	1
1	0	0	1	1
1	0	1	0	0
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

Table 3. Truth table of the second example.

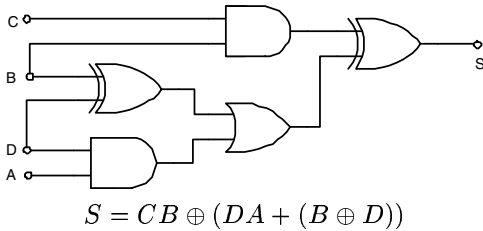


Figure 5. Graphical representation of the best circuit found for example 2.

in Table 6. Note that in this case, BPSO was not able to find feasible circuits in any of the runs performed. EBPSO found the best solution, which was a feasible circuit with 10 gates (this circuit is depicted in Figure 6). EBPSO found this solution 25% of the time, while EAPSO found it in only 2 runs and the MGA found it only once. Both EAPSO and EBPSO produced feasible circuits in at least half of their runs (50% and 55%, respectively). In contrast, the average fitness of the MGA was 35.60. Again, the best overall performer was EBPSO, with an average number of gates of 12.14 and an average fitness of 44.85. EAPSO was the second best and BPSO was the worst approach in this example. The best solution found by a human designer for this circuit had 17 gates.

3.4 Example 4

Our fourth example has 4 inputs and 2 outputs, as indicated in Table 7. In this case, BPSO, EAPSO and EBPSO used 50 particles and 4000 iterations (i.e., a total of 200,000

	BPSO	EAPSO	EBPSO	MGA	HD
# of gates of best sol.	5	5	5	5	10
Freq. of best sol.	20%	40%	55%	10%	-
Feas. circs.	85%	90%	100%	70%	-
Avg. # gates	10.40	8.25	5.90	13.70	-
Avg. fitness	30.60	32.75	35.10	27.30	-
Std. Dev.	7.1259	6.2313	1.3338	9.3982	-

Table 4. Comparison of results between our three PSO algorithms (EBPSO, EAPSO, BPSO), the multiobjective genetic algorithm (MGA), and a human designer (HD) for example 2.

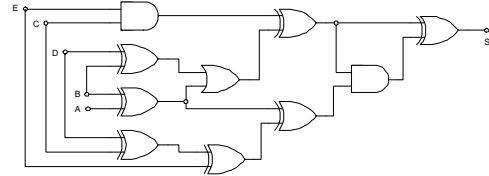


Figure 6. Graphical representation of the best circuit found for example 3.

fitness function evaluations). The MGA used a population size of 330 and 610 generations (i.e., a total of 201,300 fitness function evaluations). We show our comparison of results in Table 8. We can see in this case that our EBPSO was the only algorithm able to find the best solution in more than half of the runs (60%). The best solution found in this case is a feasible circuit with 7 gates (this solution is depicted in Figure 7). The worst contender was the MGA, which was able to find the best solution only 25% of the time. Also note that EBPSO found feasible solutions 100% of the time, having an average fitness of 46.95 with a standard deviation of 1.16. In contrast, the MGA found feasible circuits only 75% of the time, having an average fitness of 43.60. Also, the average number of gates of EBPSO was very close to the best solution found (7.75). The best solution found by a human designer for this example had 9 gates.

E	D	C	B	A	S
0	0	0	0	0	0
0	0	0	0	1	0
0	0	0	1	0	0
0	0	0	1	1	1
0	0	1	0	0	0
0	0	1	0	1	1
0	0	1	1	0	1
0	0	1	1	1	0
0	1	0	0	0	0
0	1	0	0	1	1
0	1	0	1	0	1
0	1	0	1	1	0
0	1	1	0	0	1
0	1	1	0	1	0
0	1	1	1	0	0
0	1	1	1	1	0
1	0	0	0	0	0
1	0	0	0	1	1
1	0	0	1	0	1
1	0	0	1	1	0
1	0	1	0	0	1
1	0	1	0	1	0
1	0	1	1	0	0
1	0	1	1	1	0
1	1	0	0	0	1
1	1	0	0	1	0
1	1	0	1	0	0
1	1	0	1	1	0
1	1	1	0	0	0
1	1	1	0	1	0
1	1	1	1	0	0
1	1	1	1	1	0

Table 5. Truth table of the third example.

3.5 Example 5

Our fifth example has 5 inputs and 3 outputs as indicated in Table 9. BPSO, EAPSO and EBPSO used 50 particles and 20,000 iterations (i.e., a total of 1,000,000 fitness function evaluations). The MGA used a population size of 970 and 1032 generations (i.e., a total of 1,101,040 fitness function evaluations). Our comparison of results is shown in Table 10. We can see that all the algorithms compared produced a feasible circuit with only 7 gates (depicted in Figure 8) during 20% of the runs. The worst contender was BPSO, which found this circuit only 10% of the time. Also note that the MGA had the largest percentage of feasible runs (65%) and the highest average fitness (103.95). The second best approach was EAPSO, closely followed by EBPSO. The best solution found by a human designer for

	BPSO	EAPSO	EBPSO	MGA	HD
# of gates of best sol.	*	10	10	10	17
Freq. of best sol.	0%	10%	25%	5%	-
Feas. circs.	0%	50%	55%	25%	-
Avg. # gates	*	13.80	12.15	21.40	-
Avg. fitness	28.95	43.20	44.85	35.60	-
Std. Dev.	0.9986	13.3952	13.2119	11.8028	-

Table 6. Comparison of results between our three PSO algorithms (EBPSO, EAPSO, BPSO), the multiobjective genetic algorithm (MGA), and a human designer (HD) for example 3.

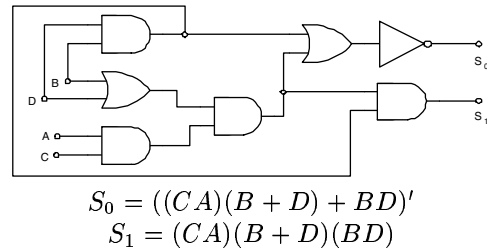


Figure 7. Graphical representation of the best circuit found for example 4.

this circuit also had 7 gates.

4 Conclusions and Future Work

We have presented a comparative study of three different encodings adopted in a PSO algorithm used to design combinational logic circuits. Our results indicate that our proposed integer encoding (called EBPSO in this paper) has a better overall performance (both in terms of consistency to reach feasible solutions and in terms of quality of the best solution found). This integer encoding was found to be superior to the use of a binary encoding and to the integer encoding proposed in [5]. Furthermore, our results were more consistent than those produced by the MGA [3], although our EBPSO was not able to improve on the quality of the best results produced by such algorithm.

As part of our future work, we are interested in exploring alternative encodings (including the possibility of using

D	C	B	A	S_0	S_1
0	0	0	0	1	0
0	0	0	1	1	0
0	0	1	0	1	0
0	0	1	1	0	0
0	1	0	0	1	0
0	1	0	1	1	0
0	1	1	0	0	0
0	1	1	1	0	0
1	0	0	0	1	0
1	0	0	1	0	0
1	0	1	0	0	0
1	0	1	1	0	1
1	1	0	0	0	0
1	1	0	1	0	0
1	1	1	0	0	1
1	1	1	1	0	1

Table 7. Truth table of the fourth example.

real numbers to encode circuits, using ideas analogous to the random keys encoding [1]). We are also interested in incorporating a multi-objective selection mechanism such as the population-based approach used in the Vector Evaluated Genetic Algorithm [13], which is adopted in our MGA [3]. This mechanism significantly improved the performance of our MGA with respect to a traditional genetic algorithm, and we expect such a behavior to be similar in the case of PSO.

Acknowledgements

The first author gratefully acknowledges support from CONACyT through project 34201-A. The second author acknowledges support from CONACyT through a scholarship to pursue graduate studies at the Computer Science Section of the Electrical Engineering Department at CINVESTAV-IPN. The third author acknowledges support from CONACyT project no. 42523.

References

- [1] J. C. Bean. Genetics and random keys for sequencing and optimization. *ORSA Journal on Computing*, 6(2):154–160, 1994.
- [2] C. A. Coello Coello, A. D. Christiansen, and A. Hernández Aguirre. Use of Evolutionary Techniques to Automate the Design of Combinational Circuits. *International Journal of Smart Engineering System Design*, 2(4):299–314, June 2000.
- [3] C. A. Coello Coello and A. Hernández Aguirre. Design of Combinational Logic Circuits through an Evolutionary

	BPSO	EAPSO	EBPSO	MGA	HD
# of gates of best sol.	7	7	7	7	9
Freq. of best sol.	30%	40%	60%	25%	-
Feas. circs.	95%	70%	100%	75%	-
Avg. # gates	10.05	13.45	7.75	13.40	-
Avg. fitness	46.95	43.55	49.25	43.60	-
Std. Dev.	4.33	8.53	1.16	8.09	-

Table 8. Comparison of results between our three PSO algorithms (EBPSO, EAPSO, BPSO), the multiobjective genetic algorithm (MGA), and a human designer (HD) for example 4.

- Multiobjective Optimization Approach. *Artificial Intelligence for Engineering, Design, Analysis and Manufacture*, 16(1):39–53, January 2002.
- [4] C. A. Coello Coello, E. Hernández Luna, and A. Hernández Aguirre. Use of particle swarm optimization to design combinational logic circuits. In A. M. Tyrell, P. C. Haddow, and J. Torresen, editors, *Evolvable Systems: From Biology to Hardware. Proceedings of the 5th International Conference, ICES 2003*, pages 398–409, Trondheim, Norway, 2003. Springer, Lecture Notes in Computer Science Vol. 2606.
- [5] X. Hu, R. C. Eberhart, and Y. Shi. Swarm Intelligence for Permutation Optimization: A Case Study on N-Queens Problem. In *Proceedings of the IEEE Swarm Intelligence Symposium 2003 (SIS 2003)*, pages 243–246, Indianapolis, Indiana, USA, 2003. IEEE.
- [6] M. Karnaug. A Map Method for Synthesis of Combinational Logic Circuits. *Transactions of the AIEE, Communications and Electronics*, 72(1):593–599, November 1953.
- [7] J. Kennedy and R. C. Eberhart. Particle Swarm Optimization. In *Proceedings of the 1995 IEEE International Conference on Neural Networks*, pages 1942–1948, Piscataway, New Jersey, 1995. IEEE Service Center.
- [8] J. Kennedy and R. C. Eberhart. *Swarm Intelligence*. Morgan Kaufmann Publishers, San Francisco, California, 2001.
- [9] S. J. Louis. *Genetic Algorithms as a Computational Tool for Design*. PhD thesis, Department of Computer Science, Indiana University, August 1993.
- [10] E. J. McCluskey. Minimization of Boolean Functions. *Bell Systems Technical Journal*, 35(5):1417–1444, November 1956.
- [11] J. F. Miller, D. Job, and V. K. Vassilev. Principles in the Evolutionary Design of Digital Circuits—Part I. *Genetic Programming and Evolvable Machines*, 1(1/2):7–35, April 2000.

E	D	C	B	A	S_0	S_1	S_2
0	0	0	0	0	1	1	0
0	0	0	0	1	1	0	0
0	0	0	1	0	1	1	0
0	0	0	1	1	1	0	0
0	0	1	0	0	1	1	1
0	0	1	0	1	1	0	1
0	0	1	1	0	1	1	0
0	0	1	1	1	1	1	0
0	1	0	0	0	1	1	0
0	1	0	0	1	1	0	0
0	1	0	1	0	1	1	0
0	1	0	1	1	1	0	0
0	1	1	0	0	1	1	1
0	1	1	0	1	1	0	1
0	1	1	1	0	1	1	0
0	1	1	1	1	1	1	0
1	0	0	0	0	0	1	0
1	0	0	0	1	0	0	0
1	0	0	1	0	0	1	0
1	0	0	1	1	0	0	0
1	0	1	0	0	0	1	1
1	0	1	0	1	0	0	1
1	0	1	1	0	0	1	0
1	0	1	1	1	0	1	0
1	1	0	0	0	0	1	0
1	1	0	0	1	0	0	0
1	1	0	1	0	0	1	0
1	1	0	1	1	0	0	0
1	1	1	0	0	1	1	1
1	1	1	0	1	1	0	1
1	1	1	1	0	1	1	0
1	1	1	1	1	1	1	0

Table 9. Truth table of the fifth example.

- [12] W. V. Quine. A Way to Simplify Truth Functions. *American Mathematical Monthly*, 62(9):627–631, 1955.
- [13] J. D. Schaffer. Multiple Objective Optimization with Vector Evaluated Genetic Algorithms. In *Genetic Algorithms and their Applications: Proceedings of the First International Conference on Genetic Algorithms*, pages 93–100. Lawrence Erlbaum, 1985.
- [14] A. Zemva, F. Brglez, and B. Zajc. Multi-level logic optimization based on wave synthesis of permissible mutation functions. In *International Workshop on Logic Synthesis (IWLS'98)*, June 1998.

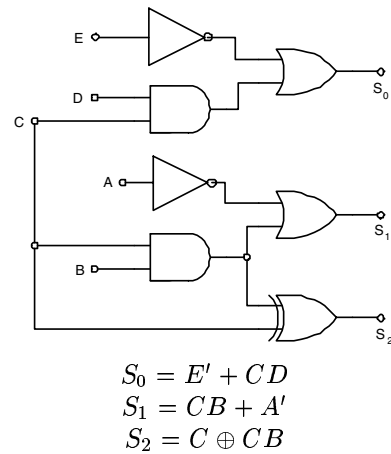


Figure 8. Graphical representation of the best circuit found for example 5.

	BPSO	EAPSO	EBPSO	MGA	HD
# of gates of best sol.	7	7	7	7	7
Freq. of best sol.	10%	20%	20%	20%	-
Feas. circs.	25%	50%	45%	65%	-
Avg. # gates	23.95	18.65	20.10	17.05	-
Avg. fitness	97.05	102.35	100.90	103.95	-
Std. Dev.	9.03	10.91	11.03	10.46	-

Table 10. Comparison of results between our three PSO algorithms (EBPSO, EAPSO, BPSO), the multiobjective genetic algorithm (MGA), and a human designer (HD) for example 5.